Design: Bidirectional Client ↔ API-App Linking + Onboarding API-Config Extension

- **Date:** 2026-06-03
- **Status:** Approved (operator, 2026-06-03)
- **Classification:** project-specific (Lava client + api-app integration; the generic launch-or-store pattern is a candidate future `vasic-digital` extraction — see §9).
- **Operator mandate (verbatim, 2026-06-03):** "we must connect both apps to each other! For example from client we MUST be able to open API app, start the API and then from API app to go back to client APP. If any of two is not installed on the System user will be taken to play store to install it! This MUST work in both directions, we MUST create good UX! API configuration screen of the onboarding wizard of the client app MUST BE properly extended to support API and work with it! Make sure we cover everything with the tests! All tests MUST produce real evidence of everything working! There cannot be any false results or bluff(s) of any kind!"

1. Goal

Wire the Lava **client** (`digital.vasic.lava.client`) and the on-device **API app** (`digital.vasic.lava.api`) so a user can move between them in BOTH directions, start the on-device API from the client, and — when either app is missing — be taken to the Play Store to install it. The client's onboarding **API configuration step** (`ApiSelectionStep`) is extended with an "On this device" section that drives this flow and, on return, auto-connects to the just-started local API.

2. Approved decisions (brainstorm Q1–Q4)

1. **Connect model = loopback auto-connect.** The API app conveys its live loopback `host:port`; the client auto-selects

`127.0.0.1:<port>`, probes `/health`, and advances onboarding. mDNS-independent.
2. **API-app behavior = auto-start + explicit "Back to Lava" button.** On a start-intent the API app auto-starts the engine, handles the notification/foreground-service permission prompt, shows running status, and presents a prominent return button.
3. **Key handoff = signature-permission ContentProvider.** The API app exposes its access key + loopback port via a `ContentProvider` guarded by a `signature`-level permission. Both apps share the keystore, so only the genuine client can read it; the key never crosses an Intent or a log (§6.H).
4. **End-to-end evidence = operator runs on the real Samsung S23 Ultra.** All hermetic tests run here with real pass/fail evidence; the two-APK cross-app Challenge tests are authored now and executed by the operator on the S23 Ultra (the emulator cannot boot on the macOS host — §6.AH-debt). Nothing is bluffed: hermetic evidence is real now; on-device evidence is real when the operator runs it.

3. Variant targeting

Cross-launch is build-variant-aware: release client ↔ release API (`digital.vasic.lava.api`), debug client ↔ debug API (`digital.vasic.lava.api.dev`). Target package IDs + the provider authority are emitted as `BuildConfig` fields in each app (package IDs are build constants, NOT §6.R secrets). The Play-Store fallback ALWAYS targets the **release** package id (debug `.dev` builds are side-loaded and have no Play listing — see §6.4).

4. New shared module: `core:applink`

An Android library depended on by both `:app` and `:api-app`, holding the cross-app contract in ONE place so the two apps cannot drift (drift between two copies of an intent contract is a classic bluff vector).

Unit	Responsibility	Depends on
`AppLinkContract`	Single source of truth for the ACTION string, EXTRA keys (`EXTRA_START_API`, `EXTRA_RETURN_TO`, `EXTRA_API_HOST`, `EXTRA_API_PORT`), and the provider authority/ permission names. Pure constants fed by `BuildConfig`.	—
`CrossAppLauncher`	Given a target package + a `PackageChecker`, returns a	`PackageChecker` (interface; real

Unit	Responsibility	Depends on
	sealed `LaunchDecision`: `Launch(intent)` when installed (`getLaunchIntentForPackage` + extras), or `StoreRedirect(marketUri, webUri)` when absent. **No side effects** — pure decision, so it is fully unit-testable.	impl wraps `PackageManager`)
`PackageChecker`	`isInstalled(pkg): Boolean` + `launchIntentFor(pkg): Intent?`. Real impl wraps `PackageManager`; tests inject a fake.	Android `PackageManager` (real impl only)

`<queries>` entries are added to BOTH apps' manifests (Android 11+ package visibility): each app queries the other's package + the `market`/`https` Play intents. Without this, `getLaunchIntentForPackage` returns null on API 30+ even when the app IS installed (silent false "not installed" → Play Store loop = a real defect this clause prevents).

5. Key ContentProvider

- **API app — `ApiKeyProvider`** (`ContentProvider`): one read-only row exposing `{ access_key, loopback_port }`. Declared in the api-app manifest with `android:exported="true"` + `android:readPermission="digital.vasic.lava.permission.READ_API_KEY"`; the permission is declared `android:protectionLevel="signature"`. Authority is variant-aware (`digital.vasic.lava.api[.dev].keyprovider`, from `BuildConfig`). Reads the key from the existing `ApiKeyStore` and the live port from the engine controller; returns an empty/ cursor when the engine is not running.
- **Client — `ApiKeyClient`**: reads the provider via `ContentResolver` using the variant-aware authority; declares the matching `uses-permission`. Returns `ApiHandoff(port, key)` or `null` (API app absent / engine not started / permission denied). The signature permission means a third-party app signed with a different key cannot read it.

6. Data flows

6.1 Direction 1 — client → API app → client (onboarding happy path)

1. Onboarding `ApiSelectionStep` "On this device" button. `OnboardingViewModel` consults `CrossAppLauncher` for the API package: installed → emit `LaunchApiApp` side effect; absent → emit `OpenPlayStore` side effect.
2. Client launches `…api[.dev]/MainActivity` with `EXTRA_START_API=true`, `EXTRA_RETURN_TO=<client pkg>`.
3. API-app `MainActivity` detects `EXTRA_START_API` → auto-starts the engine via the existing `ApiServiceStarter` → requests POST_NOTIFICATIONS if needed → on `/health` healthy, renders "API running ✓ · 127.0.0.1:`<port>`" + a prominent **"Back to Lava client"** button.
4. User taps it → API app launches client `MainActivity` with `EXTRA_API_HOST=127.0.0.1`, `EXTRA_API_PORT=<port>`.
5. Client `MainActivity` receives the return intent → routes to `OnboardingViewModel.onOnDeviceApiReturned(host, port)` → reads the key via `ApiKeyClient` → builds `Endpoint.GoApi(127.0.0.1, port, key)` → probes `/health` (existing `ConnectionService`) → auto-selects the endpoint, persists it, advances the wizard.

6.2 Direction 2 — API app → client (standalone)

API-app `MainActivity` always renders an **"Open Lava client"** button (independent of how it was launched). `CrossAppLauncher` for the client package: installed → launch; absent → Play Store (`…client` release id). When the API app was launched from the client (Direction 1), the same button additionally carries the loopback return extras.

7. Error handling (no silent failure — §6.AC telemetry on every branch)

- **Target app not installed** → Play Store via the release package id (`market://details?id=…`, with `https://play.google.com/store/apps/details?id=…` web fallback when the Play Store app is absent).
- **Debug-variant wrinkle:** `.dev` ids have no Play listing. A debug build whose counterpart `.dev` app is absent shows an explicit "side-load the Lava {API|client} dev build" message instead of a dead `market://` link.
- **Engine start failure** (API app) → API app shows an error state, emits no return result; the client side surfaces "couldn't start the on-device API" and stays on the API selection step.

- **Provider read failure / permission denied / engine not running** → `ApiKeyClient` returns null → client falls back gracefully to the existing cloud + mDNS sections (the user can still pick another API).
- **Loopback `/health` probe failure** → the existing `ApiConnectivityState.Failure` UI + "Try again" affordance.
- Each error path records a non-fatal telemetry event (`recordNonFatal`/`recordWarning`) per §6.AC, with redacted context (never the key).

8. Testing

8.1 Hermetic — real evidence on this host

- `CrossAppLauncherTest`: installed → `Launch` with the exact explicit intent (component package + `EXTRA_START_API`/`EXTRA_RETURN_TO`); absent → `StoreRedirect` with the exact `market://details?id=<release-id>` + web URI; variant selection picks the right package per `BuildConfig`. Primary assertions on the produced `Intent`/`Uri` values.
- `ApiKeyProviderTest` (Robolectric): provider returns `{key, port}` when the engine is running; empty cursor when stopped; signature-permission enforcement (a caller without the permission is denied). Primary assertion on the cursor contents + the SecurityException.
- `ApiKeyClientTest` (Robolectric): resolves the variant-aware authority; maps a row to `ApiHandoff`; null on absent provider.
- `OnboardingViewModelTest` (real UseCase + real repo + fake `PackageChecker` + real `ConnectionService` vs MockWebServer): `LaunchOnDeviceApi` → correct side effect (`LaunchApiApp` when installed / `OpenPlayStore` when not); `onOnDeviceApiReturned(127.0.0.1, port)` → reads key (fake provider reader enforcing the real contract) → builds loopback `Endpoint.GoApi` → probes the MockWebServer `/health` → selected endpoint persisted + step advanced. Primary assertion on persisted repository state + rendered state, per the feature Anti-Bluff law (no mocked UseCase).
- `ApiControlViewModelTest` (api-app, real engine controller contract via a behaviorally-equivalent fake starter): `EXTRA_START_API` → engine started → live port exposed; "Back to Lava" → correct `CrossAppLauncher` decision.
- Each test class carries a documented falsifiability rehearsal (mutation → observed failure → revert) in its KDoc + the commit Bluff-Audit stamp (Seventh Law clause 1).

8.2 On-device Challenge — authored now, operator-runs on the S23 Ultra

- `ChallengeNN_ClientLaunchesApiAppAndAutoConnects`: with BOTH real APKs installed, drive client onboarding → "On this device" → API app opens → auto-starts → "Back to Lava" → client auto-connects to `127.0.0.1:<port>` → onboarding advances. Primary assertion on the rendered post-advance onboarding state + the persisted loopback endpoint.
- `ChallengeNN_NotInstalledRedirectsToPlayStore`: API app NOT installed → tapping "On this device" fires the Play-Store intent (assert via Espresso intent capture).
- `ChallengeNN_ApiAppOpensClient`: from the API app, "Open Lava client" launches the client (installed) / Play Store (absent).
- A manual rehearsal script + attestation recorded under `.lava-ci-evidence/<run>/` (device model, Android version, both app versions, command-by-command checklist, screenshots/video). Marked honestly PENDING until the operator executes it (§6.Z gate stays open — no release/distribute until run).

9. Decoupled-Architecture note (per the Decoupled Reusable Architecture rule)

The generic "launch the target app or redirect to its store listing" primitive (`CrossAppLauncher` + `PackageChecker`) is a candidate for extraction to a `vasic-digital` submodule (e.g. `vasic-digital/AppLink`) because any two-app product would want it. It is kept in-repo as `core:applink` for this iteration because (a) the package-ID + provider-authority contract is Lava-specific, and (b) the unit is small and tightly coupled to the Lava intent contract. **Tracked as a deferred extraction TODO** with target = a future `vasic-digital/AppLink` submodule, per the rule's "documented why-not-a-submodule decision" requirement. The signature-permission provider key-handoff stays Lava-specific (it embeds Lava permission names).

10. Process / constitutional compliance

- **§6.Y** version bumps: both apps get a minor bump (new user-facing feature) as the FIRST commit of the cycle.
- **§6.S** `docs/CONTINUATION.md` updated in lockstep (new module, new feature, version bumps).
- **§11.4.113 / §6.T.3** no force-push: all pushes are fast-forward / merge-onto-latest-main; regular commit + push of all submodules + main to all upstreams.
- **§11.4.70** subagent-driven implementation.

- **§6.Z / §6.AA** no distribute until the on-device Challenges are executed on the S23 Ultra and the evidence file is present; two-stage debug-then-release distribute when that happens.
- **§6.Q** the extended `ApiSelectionStep` keeps using `Column(verticalScroll)` with bounded plain composables (no nested LazyColumn).

11. API references to verify at implementation time (§11.4.8 / §11.4.99)

The implementation phase MUST verify the current behaviour of these Android APIs against official documentation before coding (they are version-sensitive and a frequent source of stale-knowledge defects):

- Package visibility / `<queries>` (Android 11+ / API 30) — `getLaunchIntentForPackage` returning null without a `<queries>` entry.
- `ContentProvider android:protectionLevel="signature"` permission semantics + same-signing-key enforcement.
- POST_NOTIFICATIONS runtime permission (API 33+) + `foregroundServiceType=dataSync` start constraints.
- `market://details` + `https://play.google.com/store/apps/details` resolution + the "Play Store app absent" fallback.

`Sources verified <date>: <urls>` to be recorded in the implementing commits per §11.4.99.

12. Out of scope (YAGNI)

- No custom URL scheme / web deep-linking for the cross-app launch (explicit-component intents only — smaller attack surface; the existing rutracker.org VIEW filter is untouched).
- No TV (`TvActivity`) surface for the on-device API affordance in this iteration (phone/tablet onboarding only; the leanback launcher is unchanged).
- No multi-API-app support (one on-device API app per variant).